TRAVAIL INTERMÉDIAIRE N° 2

Prendre en main la modélisation

Persistance, modèle AR et ELM sur vos propres séries temporelles - travail de groupe sur GitHub

Encadrant : Cyril Voyant — cyril.voyant@minesparis.psl.eu
Support : la présentation du cours « Prévision de la ressource solaire intermittente »
Travail : individuel pour les calculs, collectif pour le dépôt GitHub
Logiciel : libre (MATLAB conseillé ; Python possible)
Remise : Semaine du 26 Octobre

En bref

- Le groupe crée **un dépôt GitHub public**, géré par un **responsable**.
- Chaque élève choisit **une série temporelle différente** sur Internet.
- Chacun prévoit sa série **un pas de temps à l'avance** avec 5 méthodes : persistance, AR, ELM, AR avec différenciation, ELM avec différenciation.
- Chacun décrit sa série (4 statistiques), calcule 5 métriques d'erreur et les reporte dans les **tableaux communs** du dépôt.
- Il ne s'agit pas encore de prévoir le GHI : c'est un entraînement à la modélisation, avant le travail collectif.

L'esprit de ce travail

Le but est d'**acquérir des compétences** et de **prendre plaisir** à modéliser. Entraidez-vous : expliquer à un autre est la meilleure façon d'apprendre.
Une question, un blocage ? N'hésitez pas à m'écrire : cyril.voyant@minesparis.psl.eu

1. Objectifs

- apprendre à construire, entraîner et évaluer un modèle de prévision, de bout en bout ;
- comprendre ce qu'apportent la persistance, l'AR et l'ELM, et l'effet d'une transformation de la série (différenciation) ;
- écrire des fonctions réutilisables (métriques, modèles) : elles serviront au travail collectif qui suit ;
- travailler à plusieurs sur un dépôt de code partagé, propre et documenté.

2. Le dépôt GitHub du groupe

2.1 Créer les comptes et le dépôt

1. **Chaque élève** crée un compte sur https://github.com (gratuit ; le programme *GitHub Education* offre des avantages aux étudiants).
2. Le groupe désigne **un responsable** du dépôt pour l'UE14.
3. Le responsable crée un dépôt **public** nommé par exemple *UE14-prevision-groupeX* : bouton *New repository*, cocher *Add a README file*, choisir une licence (*MIT License*) et un *.gitignore* (MATLAB ou Python).

4. Le responsable envoie le lien du dépôt à l'encadrant et à tous les membres.

2.2 Petit lexique

Mot	Ce que c'est
Dépôt (*repository*)	le dossier du projet hébergé sur GitHub, avec tout son historique
Commit	un enregistrement daté d'une modification, avec un message qui la décrit
Fork	votre copie personnelle du dépôt du groupe, sur votre compte
Pull request (PR)	une **demande de modification** : vous proposez vos changements, le responsable les relit puis les accepte (*merge*)
Issue	un fil de discussion pour signaler un problème ou organiser le travail
README.md	la page d'accueil du dépôt, écrite en Markdown : c'est la première chose que l'on lit
LICENSE	le fichier qui dit ce que les autres ont le droit de faire de votre code

2.3 Comment proposer une modification

Deux possibilités, au choix :

- **Par pull request** (recommandé) : faites un *fork* du dépôt, ajoutez vos fichiers dans votre copie (bouton *Add file* → *Upload files*, ou logiciel *GitHub Desktop*), puis cliquez sur *Contribute* → *Open pull request*. Le responsable relit et fusionne.
- **Par le responsable** : envoyez-lui vos fichiers ; il les dépose lui-même (un *commit* par contribution, avec un message clair).

Aucune ligne de commande n'est nécessaire : l'interface web et GitHub Desktop suffisent.

2.4 Organisation du dépôt (exemple)

```
UE14-prevision-groupeX/
├── README.md          page d'accueil (résumé, organisation, résultats)
├── LICENSE            licence du code (MIT)
├── CITATION.cff       comment citer le dépôt
├── .gitignore         fichiers à ne pas déposer
├── rapports/
│   ├── biblio/        travail n° 1 : NOM_Prenom_biblio.pdf
│   └── modelisation/  travail n° 2 : synthèse du groupe (PDF)
├── data/              données (.csv) + sources.md
├── fonctions/         métriques, persistance, AR, ELM (.m / .py)
├── codes/
│   ├── prenom1/       scripts de chaque élève + README.md
│   └── prenom2/
├── resultats/
│   ├── tableaux/      un .csv par métrique + synthese.csv
│   └── figures/       .png
└── docs/              présentation du cours (PDF)
```

Créer un dossier sur GitHub

GitHub ne garde pas les dossiers vides : un dossier se crée en y plaçant un fichier.

- Sur le site : *Add file* → *Create new file*, puis tapez le chemin avec des « / », par exemple *rapports/biblio/README.md* : les dossiers sont créés automatiquement.
- Ou *Add file* → *Upload files* : glissez-déposez un dossier entier, son arborescence est conservée.
- Le responsable peut créer toute l'arborescence dès le départ, avec un court README.md dans chaque

dossier qui explique ce qu'on doit y mettre.

2.5 Les fichiers de base et les formats à déposer

Fichier ou dossier	Format	Rôle
README.md	Markdown	page d'accueil ; chaque dossier *codes/prenom/* a aussi son petit README.md
LICENSE	texte	licence MIT (proposée par GitHub à la création du dépôt)
CITATION.cff	texte (YAML)	auteurs, titre, année : GitHub affiche alors un bouton *Cite this repository*
.gitignore	texte	exclut les fichiers inutiles (.asv, .mat temporaires, __pycache__, .DS_Store)
requirements.txt	texte	si Python : liste des bibliothèques (numpy, pandas…)
rapports/	PDF	rapports biblio (travail n° 1) et synthèse du travail n° 2 ; nommage NOM_Prenom_titre.pdf
data/	.csv (+ sources.md)	données en CSV, avec pour chaque série : lien, date de téléchargement, licence. Pas de fichier > 50 Mo
fonctions/, codes/	.m ou .py	codes commentés, un script principal par élève (main.m ou main.py)
resultats/tableaux/	.csv	une ligne par série, une colonne par modèle ; séparateur « ; »
resultats/figures/	.png	figures lisibles, axes titrés, unités

À éviter : les fichiers Word (.docx) et Excel (.xlsx) pour les codes et les résultats, les données personnelles, les fichiers très lourds.

2.6 Le README

Le README doit permettre à quelqu'un qui découvre le dépôt de tout comprendre en quelques minutes. **Enrichissez-le à chaque étape.** Il contient au minimum :

- le titre et un résumé du projet en quelques lignes ;
- les membres du groupe et le nom du responsable ;
- l'organisation des dossiers ;
- la liste des séries (qui, quoi, source, licence) ;
- comment lancer les codes (logiciel et version, script principal) ;
- les tableaux de résultats (section 6) ;
- la licence et la façon de citer le travail.

Exemple : https://github.com/cyrilvoyant/cyclostationary-forecasting-matlab

3. Choisir sa série temporelle

Chacun sa série. Coordonnez-vous : notez votre choix dans le README (ou dans une *issue*) avant de commencer, pour éviter les doublons.

Domaine	Exemples de séries	Où les trouver
Énergie	consommation électrique, production éolienne ou solaire, prix de l'électricité	RTE éCO2mix, ENTSO-E Transparency Platform, OpenDAta EDF Corse, etc…
Météorologie	température, vent, humidité, pression	Météo-France (meteo.data.gouv.fr), Open-

		Meteo
Économie	prix, indices, taux de change, production industrielle	INSEE, Eurostat, FRED
Environnement	qualité de l'air (NO_2, PM10, ozone)	Geod'air, associations régionales (Atmo)
Hydrologie	débit d'une rivière	Hub'Eau (hydrométrie)
Transport et usages	trafic routier, vélos en libre-service, fréquentation	data.gouv.fr, portails open data des villes
Divers	jeux de données variés	Kaggle, UCI Machine Learning Repository

Contraintes sur la série

- pas de temps **régulier** (par exemple horaire) ; comblez les éventuels trous et expliquez comment ;
- assez de valeurs pour le découpage ci-dessous : **au moins 18 000 valeurs successives** (par exemple 2 ans ou plus au pas horaire) ;
- notez la **source** et la **licence** des données dans le README.

3.1 Découpage chronologique

Période	Nombre de valeurs	À quoi elle sert
Apprentissage	de 8 000 à 20 000 (1 an horaire = 8 760)	estimer les coefficients des modèles
Validation	au moins 2 000, au plus 20 000	choisir p (AR), L et le tirage (ELM)
Test	de 8 000 à 20 000	mesurer les erreurs, une seule fois, à la fin

Les trois périodes se suivent dans le temps, dans cet ordre, sans mélange : on ne prévoit jamais le passé avec le futur.

4. Les modèles (prévision à un pas de temps)

On note x_t la valeur de la série à l'instant t et $\hat{x}_{t+1}$ sa prévision. Toutes les prévisions portent sur le **pas de temps suivant** (horizon 1). Voir les chapitres 3, 5 et 6 de la présentation.

4.1 Persistance

$$\hat{x}_{t+1} = x_t$$

« Demain sera comme aujourd'hui. » C'est la référence : un modèle qui ne fait pas mieux n'apporte rien.

4.2 Modèle AR(p)

On centre la série avec la moyenne μ calculée sur l'apprentissage, puis on estime les coefficients w par moindres carrés :

$$\hat{x}_{t+1} = \mu + w_1(x_t - \mu) + w_2(x_{t-1} - \mu) + \ldots + w_p(x_{t-p+1} - \mu)$$

Choisissez p (par exemple entre 1 et 24) avec l'erreur sur la **validation** ou en mode Filter en utilise le PACF (voir cours), en gardant le plus petit p presque aussi bon que le meilleur (parcimonie).

4.3 ELM

Mêmes entrées que l'AR. Couche cachée de L neurones à poids aléatoires fixés, poids de sortie estimés par ridge ($\alpha = 10^{-3}$). Testez quelques valeurs de L (5 à 500) sur la validation. Faites 20 tirages pour chaque cas et gardez celui qui a la plus petite erreur de validation.

4.4 Avec différenciation

Beaucoup de séries ont une tendance (elles montent ou descendent dans le temps). La **différenciation** la supprime : on travaille sur les variations plutôt que sur les valeurs.

1. **Transformer** : $y_t = x_t - x_{t-1}$ (la variation entre deux pas de temps).
2. **Prévoir** la variation $\hat{y}_{t+1}$ avec l'AR ou l'ELM, appris sur la série y.
3. **Revenir à la série de départ** (« recolorer ») : $\hat{x}_{t+1} = x_t + \hat{y}_{t+1}$.
4. **Calculer les erreurs sur x**, jamais sur y.

> **Exemple chiffré**
>
> Série : x = 100 ; 104 ; 103 ; 107. Variations : y = 4 ; −1 ; 4.
>
> Si le modèle prévoit $\hat{y} = 1{,}5$ pour la variation suivante, alors $\hat{x} = 107 + 1{,}5 =$ **108,5**.
>
> Remarque : prévoir $\hat{y} = 0$ revient exactement à la persistance. La différenciation demande donc au modèle de faire **mieux que la persistance**.

Au total, **5 prévisions par série** : persistance, AR, ELM, AR avec différenciation, ELM avec différenciation. Si la grandeur ne peut pas être négative, remplacez les prévisions négatives par 0.

5. Mesurer l'erreur et la variabilité

Toutes les erreurs se calculent **sur le test**, sur la série x. On note $e_t = \hat{x}_t - x_t$ l'erreur de prévision, N le nombre de valeurs du test et $\bar{x}$ la moyenne de x sur le test.

Métrique	Formule	Ce qu'elle dit
nMBE	$(1/N)\ \Sigma\ e_t\ /\ \bar{x}$	le biais : > 0 si le modèle sur-estime, < 0 s'il sous-estime. Si |nMBE| > 1 %, il y a sans doute un problème
nMAE	$(1/N)\ \Sigma\ \lvert e_t\rvert\ /\ \bar{x}$	l'erreur moyenne, en % de la moyenne
nRMSE	$\sqrt{[(1/N)\ \Sigma\ e_t^2]}\ /\ \bar{x}$	comme nMAE, mais punit plus les grosses erreurs
R^2	$1 - \Sigma\ e_t^2\ /\ \Sigma\ (x_t - \bar{x})^2$	part de la variance expliquée ; 1 = parfait
$NICE^{\Sigma}$	moyenne pour k = 1, 2, 3 de L^k(modèle) / L^k(persistance), avec $L^k = [(1/N)\ \Sigma\ \lvert e_t\rvert^k]^{1/k}$	< 1 : meilleur que la persistance ; la persistance vaut 1 par construction

5.1 Décrire sa série : quatre statistiques

Avant de modéliser, décrivez votre série sur l'apprentissage, **brute** (x) et **différenciée** ($y = x_t - x_{t-1}$), avec quatre statistiques :

Statistique	Formule	Ce qu'elle dit
Moyenne	$m = (1/N)\ \Sigma\ x_t$	le niveau moyen ; proche de 0 pour la série différenciée si la série n'a pas de tendance
Écart-type	$\sigma = \sqrt{[(1/N)\ \Sigma\ (x_t - m)^2]}$	l'amplitude des variations
Asymétrie (*skewness*)	$(1/N)\ \Sigma\ (x_t - m)^3\ /\ \sigma^3$	0 : symétrique ; > 0 : quelques valeurs très fortes (pics) ; < 0 : quelques valeurs très faibles

Aplatissement (*kurtosis*)	$(1/N)\ \Sigma\ (x_t - m)^4 / \sigma^4$	3 pour une loi normale ; > 3 : valeurs extrêmes fréquentes (sauts brusques)

Comparez brute et différenciée : la différenciation réduit-elle l'écart-type ? Fait-elle apparaître des sauts (kurtosis élevé) ? Pour le GHI, la présentation définit aussi un indicateur de variabilité plus fin, le sCV, qui utilise le ciel clair (chapitre 4).

5.2 Des fonctions réutilisables

Écrivez **une fonction par métrique** et placez-les dans *fonctions/* : elles serviront à tout le groupe pour le travail collectif. Par exemple, en MATLAB :

```
function v = nrmse(x, xhat)
%NRMSE  RMSE normalisé par la moyenne de la mesure (en %).
e = xhat - x;
v = 100 * sqrt(mean(e.^2)) / mean(x);
end
```

Faites de même pour nmbe, nmae, r2, nice_sigma (qui reçoit aussi la prévision de persistance) et les quatre statistiques (en MATLAB : mean, std, skewness, kurtosis).

6. Les tableaux de résultats

Chacun ajoute **sa ligne** dans les tableaux communs, placés dans le README et dans *resultats/* (fichiers .csv). **Tableau de synthèse** (nRMSE sur le test) :

Série (élève)	Domaine, pas	N appr. / N test	Pers.	AR	ELM	AR diff.	ELM diff.
Série 1 (Prénom)	…	… / …	… %	… %	… %	… %	… %
…	…	…	…	…	…	…	…

Et un **tableau des statistiques** : une ligne par série, avec moyenne, écart-type, asymétrie et kurtosis, pour la série brute et pour la série différenciée (8 colonnes).

Puis **un tableau par métrique** (nMBE, nMAE, nRMSE, R^2, $NICE^{\Sigma}$), avec une ligne par série et les cinq mêmes colonnes de modèles.

Pour chaque série, ajoutez dans le README **un court paragraphe d'analyse** : le modèle le plus performant, l'effet de la différenciation, le lien entre les statistiques de la série (écart-type, kurtosis de la série différenciée) et le gain par rapport à la persistance.

7. Livrables et évaluation

Livrables

- le lien du dépôt GitHub **public**, envoyé à l'encadrant ;
- un README complet et à jour ;
- les codes commentés de chacun (*codes/prenom/*) et les fonctions communes (*fonctions/*) ;
- les tableaux de résultats (.csv) et quelques figures (série, prévisions, erreurs) ;
- les fichiers de base : LICENSE, CITATION.cff, .gitignore (et requirements.txt si Python) ;
- les rapports bibliographiques du travail n° 1 (PDF) dans *rapports/biblio/*.

Critères : clarté du dépôt et du README ; codes lisibles et relançables par un autre membre ; justesse des résultats (par exemple $NICE^{\Sigma} = 1$ pour la persistance) ; qualité des analyses ; qualité du travail collectif (coordination, pull requests).

8. Et ensuite ?

Le travail collectif suivant portera sur la **prévision du GHI à Ajaccio**. Comme tous les codes seront sur GitHub, chacun pourra réutiliser ceux des autres. Les codes seront **validés par validation croisée** (chaque code est testé par un autre membre, sur d'autres données), puis le groupe retiendra les plus performants et les plus frugaux.

9. Questions fréquentes

Question	Réponse
Ma série a des trous.	Comblez-les (par exemple par interpolation linéaire pour les petits trous) et dites comment dans le README.
Mon ELM fait moins bien que la persistance.	C'est possible, et c'est un résultat ! Vérifiez L, la normalisation des entrées et le découpage, puis commentez.
Faut-il un pas horaire ?	Non : tout pas de temps régulier convient, si le nombre de valeurs est suffisant.
Puis-je utiliser des boîtes à outils ?	Oui, mais vous devez comprendre et expliquer ce qu'elles font. Coder soi-même AR et ELM est très formateur.
Je ne connais pas Git.	Utilisez l'interface web de GitHub ou GitHub Desktop : aucune ligne de commande n'est nécessaire.